



MRSA: A multi-mode reconfigurable systolic array architecture based on channel-wise stationary dataflow for CNNs

Xiaojun Zhang^{ID}, Xin Song, Shuguang Liu, Yue Liu, Jian Gao^{ID}*

Shandong University of Science and Technology, No. 579 Qianwangang Road, Qingdao, 266590, Shandong, China

ARTICLE INFO

Keywords:

CNN accelerator
Reconfigurable systolic array
Multi-mode
Channel-wise stationary dataflow

ABSTRACT

Convolutional neural networks (CNNs) exhibit increasing architectural depth and substantial heterogeneity in input feature map dimensionality, presenting significant challenges for efficient hardware acceleration. Existing dataflows often have difficulty maintaining high processing unit utilization and low computational latency while ensuring compatibility. To address this issue, we designed a channel-wise stationary (CS) dataflow for CNN accelerators, which can be compatible with arbitrary convolutional kernel sizes and effectively reuses ifmaps, alleviating access bandwidth bottlenecks. Based on this, we proposed a multi-mode reconfigurable systolic array (MRSA) architecture. By splitting data in the channel dimension and extending the systolic array (SA) units in the column dimension, the SA can support various sizes of input tensors in different modes without incurring extra memory bandwidth overhead. Accurate scheduling of processing units enables pipeline cyclic splicing and latency overlapping, enhancing the computational unit utilization. Experiments show that the MRSA architecture based on CS dataflow achieves average processing unit utilization rates of 86.5%, 94.9%, and 98.8% in the convolutional layers of AlexNet, YOLOv3-Tiny, and VGG16, respectively.

1. Introduction

The escalating computational complexity and depth of modern Convolutional Neural Networks (CNNs) have necessitated the development of specialized hardware accelerators to meet real-time inference requirements [1–4]. Among various acceleration paradigms, systolic arrays (SAs) have emerged as a predominant architectural choice for CNN accelerators due to their ability to significantly reduce memory bandwidth requirements through efficient data reuse patterns.

Contemporary SA implementations for CNNs predominantly employ the NCWH storage format, necessitating transformation through the `im2col+gemm` methodology. This approach converts convolution operations into general matrix multiplication, thereby enabling batch-level parallelism that can be efficiently mapped onto systolic arrays. The seminal work by Chen et al. introduced the Row Stationary (RS) dataflow [1,2], which has profoundly influenced subsequent research in dataflow optimization and SA architectures. Subsequent innovations have led to the development of Output Stationary (OS) [5] and Weight Stationary (WS) dataflows [6], which maximize local reuse of weights and input activations, respectively. However, as the size of feature maps and the number of channels increase, the intermediate data storage requirements of WS and OS grow linearly, leading to a significant rise in on-chip storage occupancy. To alleviate storage pressure, Sungju Ryu et al. [7] proposed the Channel Stationary dataflow. This dataflow

adopts the computational structure of Separable Convolution, which divides a conventional convolution operation into Depthwise Convolution and Pointwise Convolution. This method can significantly reduce the number of parameters, thereby lowering the demand for storage resources and bandwidth, making it a resource-friendly convolutional network architecture.

Zhao et al. conducted comprehensive research on the performance of SAs under different dataflow strategies and developed a lightweight CNN accelerator architecture based on this research [8]. Existing dataflow methods typically reduce latency by partitioning operations into smaller batches and then allocating these batches to processing elements (PEs) or processing clusters. Huang et al. proposed a real-time configurable CNN accelerator that achieves cycle-accurate control of PEs through reconfigurable sequence mapping tables [9]. For specific application scenarios, researchers have also developed customized solutions. For example, Chua et al. designed an SA for convolution kernel matching in MNIST inference [10], and this array achieves high energy efficiency. However, the SA design of this specialized architecture only supports convolution kernels of fixed sizes and cannot be reconfigured to adapt to networks of multiple sizes, which affects its versatility.

Innovations in the field of architectural optimization continue to advance. Parmar et al. improved the systolic ring architecture, which not only reduces the total latency of convolution operations but also

* Corresponding author.

E-mail address: JianGao@sdust.edu.cn (J. Gao).

eliminates redundant external memory accesses for input feature maps (ifmaps) [11]. Véstias et al. proposed a configurable architecture for resource-constrained FPGAs. This architecture adopts hybrid fixed-point quantization and image batching techniques to effectively reduce bandwidth requirements [12]. However, the fixed parallelism mode of this architecture cannot adapt to the dynamic resource demands of different convolutional layers, leading to a significant decline in PE utilization in complex network architectures.

Instruction-based acceleration methods have also gained widespread attention. Li et al. proposed an instruction set architecture for ResNet18 and MobileNetV1. This architecture integrates specialized instruction mechanisms to enable flexible dataflow control [13]. In the FPGA field, Trio Adiono et al. detailed a low-cost accelerator for YOLOv3-Tiny. This accelerator achieves excellent frame rates and low latency through asynchronous operation of data preparation and computation using dataflow paradigms [14]. However, this design does not eliminate padding calculations. Instead, it uses a dedicated zero-padding module, which causes the padding area to be included in the computation, resulting in a waste of computing resources. Later, Liu et al. proposed an optimization framework based on loop blocking, loop swapping and loop unrolling techniques. Through adaptive parallelism adjustment, this framework achieves a PE utilization rate of 89.1% for ResNet-152 [15]. Despite these advances in existing research, the critical issue of redundant zero-padding computations in small-sized convolution kernels remains unresolved.

Overall, existing research on CNN hardware acceleration has carried out multi-dimensional exploration around the core architecture of SAs. In the field of dataflow innovations, from WS and OS to Channel Stationary (CS), and further to the design of customized architectures tailored for lightweight requirements and FPGA resource-constrained scenarios, relevant research in this domain has achieved significant breakthroughs in reducing memory bandwidth requirements, improving PE utilization, and optimizing inference latency and energy efficiency. However, current solutions still face key bottlenecks in multiple aspects. Specialized architectures are often limited by fixed convolution kernel sizes and struggle to adapt to multi-size networks, resulting in insufficient versatility. Fixed parallelism modes cannot respond to the dynamic resource demands of different convolutional layers, which easily leads to a significant decline in PE utilization in complex networks. The issue of redundant zero-padding computations in small-sized convolution kernels has not been effectively resolved, and there is also a lack of mature solutions for coordinating compatibility between processing units. Based on this, this paper intends to propose a new reconfigurable architecture. By dynamically adapting to the differentiated computational demands of different CNN layers, it systematically overcomes the aforementioned challenges of versatility, resource adaptability, and redundant computations to further improve the comprehensive performance of CNN hardware acceleration.

The main contributions of this paper are as follows:

(1) To address the cross-architecture compatibility challenges in CNN accelerators while minimizing redundant data movement, we designed a Channel-wise Stationary (CS) dataflow method, which utilizes the NWHC storage format and eliminates the need for the traditional im2col transformation at the algorithmic level. This design not only helps alleviate memory bandwidth pressure but also allows for more efficient skipping of zero-padding computations, further optimizing the computational process.

(2) We implemented a channel-wise partitioning approach using CS dataflow to decouple input data segmentation from convolution kernel dimensions, eliminating kernel-size-dependent partitioning. Additionally, the simplified control logic only increases the loop count for the corresponding dimension when handling different kernel sizes. Moreover, to improve the utilization of PE units, we have optimized the computation process of the systolic array by employing loop splicing and latency overlapping techniques. Precise scheduling minimizes pipeline stalls, significantly boosting computational efficiency.

(3) To achieve compatibility with input tensors of various sizes, we propose a multi-mode compatible reconfigurable systolic array (MRSA) architecture. This architecture enhances the utilization of processing units by enabling flexible connections of multiple systolic arrays across different dimensions via mode switching, while avoiding the consumption of additional storage bandwidth. Moreover, for different types of network models, we can achieve a reconfigurable accelerator design by constructing different systolic array units and connection strategies, further enhancing its adaptability and scalability.

2. Hardware architecture and dataflow

2.1. Architecture

As illustrated in Fig. 1, the overall accelerator integrates a computational module, controllers, and a storage module, with the computational module centered on a systolic array composed of multiple processing elements (PEs). Leveraging microinstructions stored in program memory, it orchestrates computations for convolutional layers, fully connected layers, softmax layers, and more. A two-level cache structure enables efficient data access by fetching ifmap and weights from DDR4 over the system bus, staging them in the cache for processing, and ultimately writing the results back to DDR4.

2.2. Channel-wise stationary dataflow

Given a convolutional layer with the following parameters: input channel C , output channel N , ifmaps size H^2 , and kernel size R^2 , we represent the tensor of partial sum ($psum$), ifmap, and weights as $psum[N][H][H]$, $ifmap[C][H][H]$, and $W[N][C][R][R]$, respectively. Assuming that we have an SA comprising I rows and K columns, where the weights are shared among the K columns and the $psum$ is accumulated across the I rows. The innermost loop of the CS can be described by Eq. (1), and the variables and the sequence of the outer loops are listed below

$$psum[n][x][y] = \sum_{c=f_x \times I}^{(i_c+1) \times I-1} ifmap[c][x+f_x][y+f_y] \times W[n][c][f_x][f_y] \quad (1)$$

$$0 \leq x, y < H, 0 \leq f_x, f_y < R, 0 \leq n < N, 0 \leq i_c < \lceil \frac{C}{I} \rceil$$

Before the SA begins operations, the ifmap and weights of size $[I, K]$ need to be preloaded into the array at the same time. The entire kernel with I channels is then loaded and processed while the ifmap remains stationary in the PEs. This allows for full ifmap reuse for H^2 times. By leveraging the pipeline, the PE array can unfold the innermost loop in parallel, executing $I \times K$ MACs per cycle. The complete loop for a single layer can be expressed as Eq. (2)

$$O[n][x][y] = \sum_{i_c=0}^{\lceil \frac{C}{I} \rceil-1} \sum_{f_x=0}^{R-1} \sum_{f_y=0}^{R-1} psum[n][x][y] \quad (2)$$

Fig. 2 illustrates the dataflow mapping approach, and the optimization of data reading and reuse is described in the following. The reuse of the ifmap in the column dimension can be achieved via the shift operation of the SA. By reading data only once in the column dimension and reusing the ifmap through internal SA shifts, on-chip memory consumption is reduced. This avoids the overhead from matrix mapping's additional control logic and first-in-first-out (FIFO) buffers. In the row dimension, data reuse occurs between neighboring rows of the output feature map (ofmap). Part of the ifmap is loaded from off-chip memory into a cache, forming an ifmap_buffer for reuse in adjacent rows of the ofmap. This prefetching mechanism enables data reuse of the CS dataflow in the row dimension, allowing all ifmap and corresponding weight operations to be completed with a single read from off-chip memory, significantly reducing bandwidth requirements.

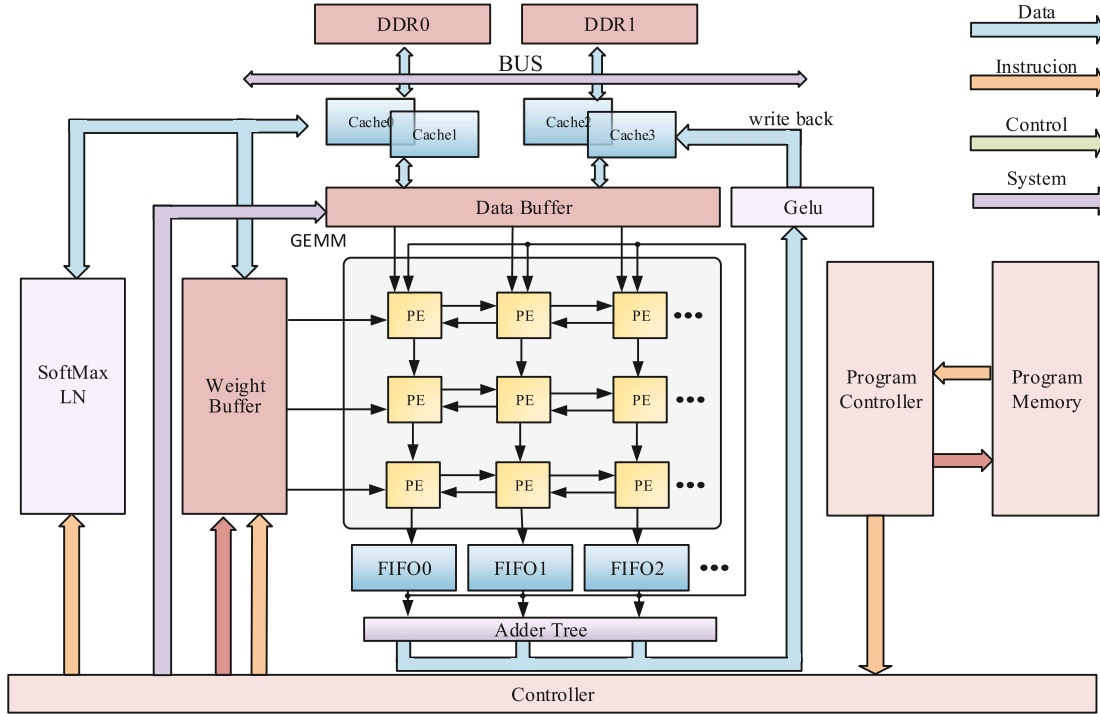


Fig. 1. Accelerator architecture.

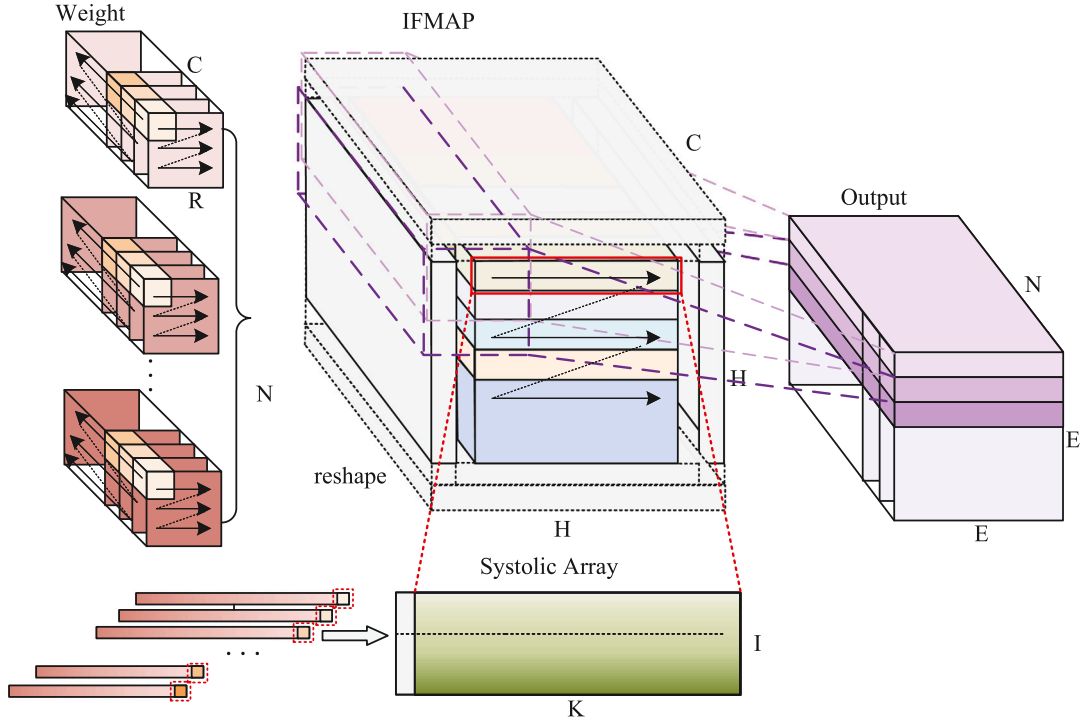


Fig. 2. Schematic diagram of channel-wise stationary dataflow.

In Fig. 2, when the padding $P = 1$, the light gray regions represent zero-padded values, and the top row corresponds to the multiply-add operation with the first row of weights. According to the CS dataflow, instead of padding the ifmap rows, the second and third rows of the ifmap are directly multiplied by the corresponding weights. In hardware implementations, the state machine skips padding by detecting invalid rows during state transitions, thereby eliminating one loop. This

approach directly avoids padding the top and bottom rows, thereby reducing computational resource consumption.

2.3. Architectural adaptability of CS dataflow for variable-sized convolutional operators

In convolution operations, kernel sizes can vary significantly across layers, ranging from small sizes like 3×3 to larger sizes like 11×11

```

L1:  for(x=0;x<H;x++){// row in feature map loop
L2:    for(y=0;y<H/K;y++){// column in feature map loop
L3:      for(fx=0;fx<R;fx++){// row in kernel loop
L4:        for(c=0;c<C;c++){// channel in feature map loop
L5:          for(fy=0;fy<R;fy++){// column in kernel loop
L6:            for(n=0;n<N;n++){// group in feature map loop
              PE_Column0_Sum[n][x][y*K]+=Ifmap[c][x+fx][y*K+fy]* W[n][c][fx][fy];
              PE_Column1_Sum[n][x][y*K+1]+=Ifmap[c][x+fx][y*K+1+fy]* W[n][c][fx][fy];
              .....
              PE_ColumnK-1_Sum[n][x][(y+1)*K-1]+=Ifmap[c][x+fx][(y+1)*K-1+fy]* W[n][c][fx][fy];
            }// end of group in feature map loop
          }// end of column in kernel loop
        }// end of channel in feature map loop
      }// end of row in kernel loop
    }// end of column in feature map loop
  }//end of row in feature map loop

```

Fig. 3. Pseudocode for systolic array operation.

or beyond. The channel-wise stationary dataflow splits data along the channel dimension, effectively mitigating the impact of diverse kernel row and column sizes on input data partitioning. As shown in Fig. 3, PE_Column_Sum represents the cumulative sum of each column in the SA. Changes in kernel size merely increase the loop iterations of L3 and L5 without introducing extra control logic or resource overhead.

Moreover, when traversing the convolution kernel, the same ifmap must be loaded into the array R^2 times. This not only degrades computational efficiency but also elevates the accelerator's bandwidth demands. However, in the PE units designed in this work, the integration of a left-shift operation eliminates the need to reload the input tensor corresponding to the L5 loop into the systolic array, avoiding R repeated data accesses. This dataflow maps the column dimension of the kernel to the number of shifts, the row dimension into the number of loops, and the stride into the shift length, enabling seamless compatibility with kernels of varying sizes.

2.4. Multi-mode reconfigurable systolic array architecture

To enhance the parallelism of the accelerator and thereby reduce processing latency, we propose an MRSA architecture based on the CS dataflow. This architecture enhances the versatility of the SA for different CNNs and various layers of the same network model, as illustrated in Fig. 4. Taking the VGG16 network as an instance, the smallest ofmap size in its convolutional layer is 14×14 . We set I to 32 and M to 14, where M represents the column dimension of the PE Tile (PET).

To enhance the compatibility of the SA across different layers, we can adapt the SA to various layers through configurable operating modes. In Fig. 4(a), the 8 PETs can be divided into 8, 4, and 2 groups, corresponding to partitioning the ifmap and weights into 8, 4, and 2 parts in the channel dimension. The PETs in each group are concatenated along the column dimension, with each group completing the convolution operation represented by one color block in Fig. 4(b). This approach ensures compatibility with different ofmap sizes such as 14, 28, and 56. Assuming the SA column size after PET splicing is $n \times M$, i.e., $K = n \times M$, where n is the number of PETs in each group. When $E = t \times K, t > 1$, the left part of the yellow dotted line in Fig. 4(b) is computed first, before traversing according to the channel dimension. When $t = 1$, each group of the PETs in Fig. 4(a) works in parallel, with each group corresponding to a different channel in the same row of the ifmap. Each group of the PETs processes a portion of the ifmap after channel dimension slicing in Fig. 4(b), and the final computation result is obtained through an adder tree.

Additionally, to enhance the compatibility of the SA with different network models, we reconfigured the PET design to meet the computational requirements of various types of networks. For irregular network models, such as AlexNet, where the ofmap sizes include dimensions like 13 and 27, the computational requirements of different layers can be met by partial padding.

2.5. Pipeline optimization

The process of the SA can be divided into four steps, loading inputs into the PE, loading weights from the left side, shifting local data, and loading a new set of weights. By cycling these steps according to dimensions such as convolution kernel size, the points in a row of the ofmap can be computed. To improve throughput and PE utilization, this MRSA architecture pipelines and optimizes these four steps, achieving loop pipelining and latency overlap via precise scheduling of each PE.

When $R = 2$ and $H = 4$, the operation of a 2×3 SA is illustrated in Fig. 5. Before pipelining optimization, the first ifmap element and the first weight are input simultaneously into the upper-left PE at cycle #0, and subsequent data is loaded in each clock cycle. At cycle #5, the ifmap shifts, requiring all columns to complete their computations before new data is loaded (cycles #3-4). During ifmap switching at cycle #10, the pipeline must be drained. After pipelining optimization, starting at cycle #0, the ifmap is loaded into the first column, and weights are loaded into the first PE. Computation continues until cycle #2, when the first row computation completes. At cycle #3, the ifmap $I_{0,0,1}$ shifts left to compute with $W_{0,0,1}$. The optimization allows for precise PE scheduling without waiting for column completion before loading new data. The ifmap shifts occur at cycles #3 and #4, with new weights loaded at cycle #6, achieving cycle stealing and latency hiding. Optimizing control logic and adding cache units ensures precise PE scheduling, maintaining continuous PE operation and saving clock cycles, except for the initial data loading and final pipeline draining phases.

The systolic array's computational latency (omitting zero-padding effects) is formalized through architectural analysis. Before pipeline optimization, the required clock cycles for computing one row of the ifmap are shown in Eq. (3), where the size of the SA is $I \times K$ and PE_{delay} represents the clock cycles consumed by a single PE.

$$T_{clk} = (PE_{delay} \times I + N + K) \times R^2 \times \frac{C}{I} \quad (3)$$

After pipeline optimization, the required clock cycles are calculated as Eq. (4), eliminating the $PE_{delay} \times I$ clock cycles required to reload the

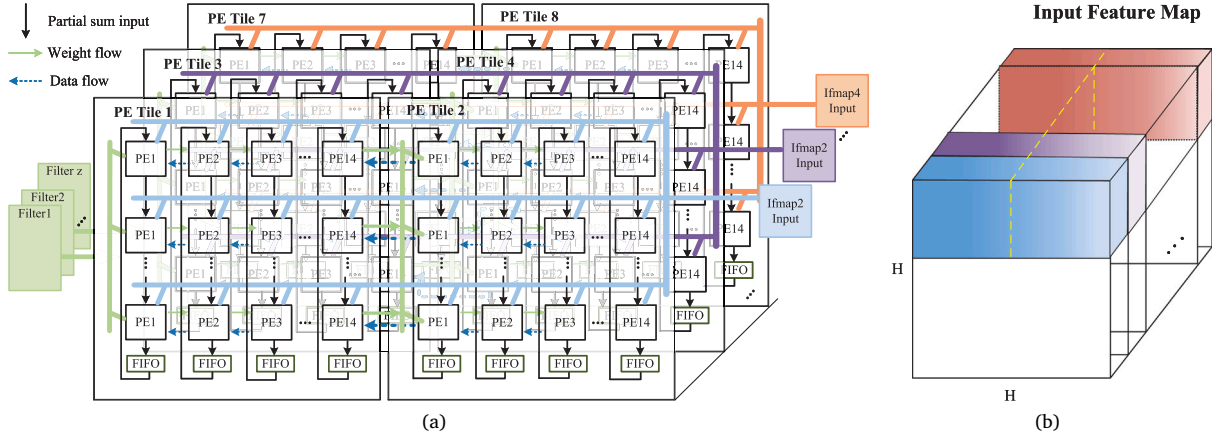


Fig. 4. Multi-mode reconfigurable systolic array. (a) An example of a connection strategy. (b) Input feature map splitting.

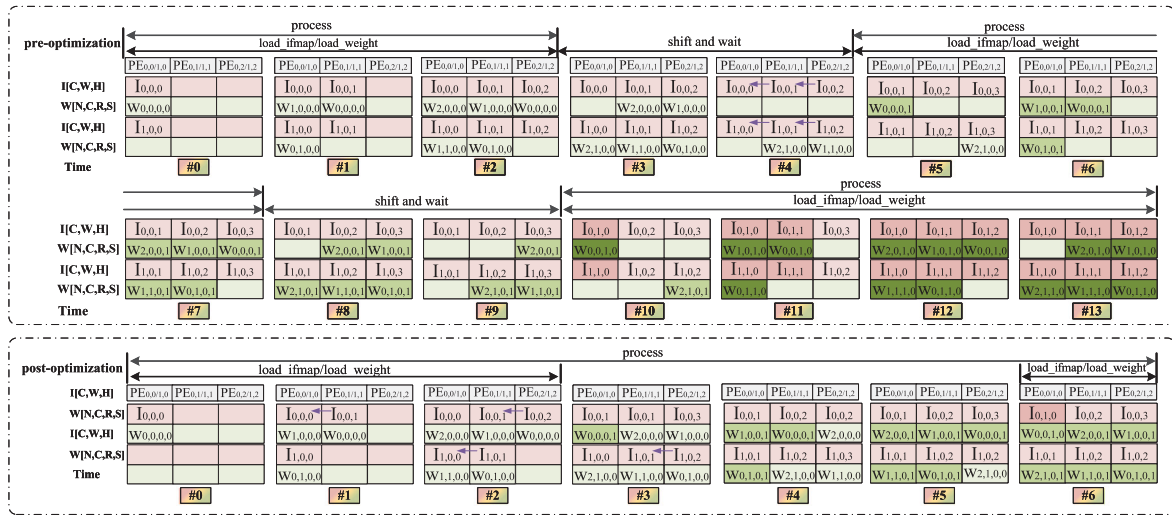


Fig. 5. Pipeline optimization of systolic array.

input weights after the shift, and there is no pipeline emptying interval K when the ifmap is shifted.

$$T'_{clk} = PE_{delay} \times I + N \times R^2 \times \frac{C}{I} \quad (4)$$

In Fig. 5, although the small-sized SA results in only a few saved clock cycles during the loading of the ifmap and the waiting period after the ifmap shift, Eq. (4) still needs to account for the cycles of batch processing, rows of ofmap, and rows of the convolution kernel, multiplied by a larger coefficient. For larger-sized SAs, the percentage of K in the overall equation increases, making the pipelining optimization crucial for enhancing the accelerator's overall performance.

3. Experiment results

To evaluate the proposed MRSA, we employ three CNN benchmarks, AlexNet, VGG16, and YOLOv3-Tiny. The experiment adopts an RTL-based hardware design approach, with all three networks using a 16-bit fixed-point format. We demonstrate their performances in terms of layer latency, throughput, hardware resource consumption, and compare with other FPGA-based accelerators.

Since the ofmaps of the AlexNet network have sizes such as 13×13 and 27×27 , this paper uses four slices of 14×32 SAs for testing, and eight slices of 14×32 and two slices of 13×32 SAs for VGG16 and YOLOv3-Tiny, respectively. For AlexNet, it combines multiple kernel

sizes and varying stride lengths, our scheme demonstrates strong compatibility. Fig. 6 shows the latency and PE utilization of convolutional layers in AlexNet and VGG16, compared with those reported in other studies. All the convolutional layers, except the first one, tile the channels in the row dimension of the SA. Compared to Li J.[17], our approach achieves high PE utilization for all but the first layer. Lee S.[24] still shows a significant advantage in computational efficiency due to the optimization strategy of double MAC.

Table 1 presents the performance comparison of AlexNet, VGG16, and YOLOv3-Tiny networks deployed on various FPGA accelerators. For AlexNet, compared to the schemes of Shan J.[16] and Li J.[17], our approach reduces latency by 2.02× and 2.32×, respectively, while achieving higher throughput performance without a proportional increase in DSP resource consumption. This is attributed to the higher utilization of the SA, which minimizes pipeline intervals. We also provide a performance comparison for the three networks using the same four PETs(CS-4PETs) in Table 1. Despite varying R sizes among the networks, our approach, which utilizes an instruction set for configurable control flow, achieves strong acceleration performance across all networks. Specifically, under the CS-4PETs configuration, the PE utilization rates for VGG16 and YOLOv3-Tiny are 98.8% and 88.1%, respectively, under these conditions. Due to the MRSA acceleration strategy, our approach maintains good computational structure compatibility with the same PET size and number of pieces, ensuring

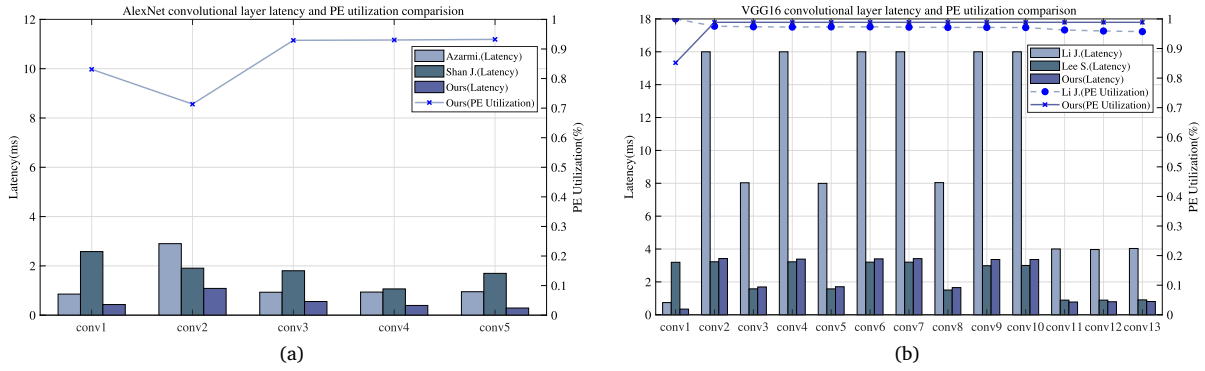


Fig. 6. Alexnet and VGG16 convolutional layer latency and PE utilization comparison.

Table 1

Comparison of the performance of different FPGA accelerators for Alexnet, VGG16 and YOLOv3-Tiny.

Network	Approach	Platform	Freq (MHz)	Logical elements	DSPs	BRAM (kb)	Precision	Latency (ms)	Throughput (GOPS)	GOPS/DSPs /Freq.(10 ⁻³)	Normalized DSP efficiency	Power (W)	GOPS/W
AlexNet	Shan J. [16]	UltraScale+	250	–	–	–	Fixed 16	9.5	229.8	–	–	–	–
	Li J. [17]	VC709	200	118K	664	467(36 kb)	Fixed 8/16	6.3	220	1.60	0.160	9.61	22.9
	Zhang Y. [18]	ZCU102	200	324K	528	1652(18 kb)	Fixed 8	–	280.3	2.65	0.132	6.1	45.95
	^a CS-4PETs	Stratix10	150	125K	908	1508(M20k)	Fixed 16	2.9	466.4	3.42	0.171	7.15	65.23
VGG16	Ma Y. [19]	Arria10	200	138K	1518	2232(M20k)	Fixed 16	43.2	715.9	2.36	0.235	–	–
	Li J. [17]	VC709	200	433.2K	664	467(36 kb)	Fixed 8/16	133.5	230.1	1.73	0.173	9.61	23.9
	Huang M. [20]	ZCU102	214	216K	2060	963(18 kb)	Fixed 2/4/8	39.5	784.2	1.78	0.089	–	185.4
	Ours	Stratix10	150	219K	1805	2225(M20k)	Fixed 16	28.3	1074.0	3.97	0.198	14.31	75.12
	^a CS-4PETs	Stratix10	150	125K	908	1508(M20k)	Fixed 16	56.2	536.8	3.94	0.197	7.34	73.13
YOLOv3-Tiny	Ahmad A. [21]	VC707	200	48.5K	2304	141(18 kb)	Fixed 18	–	460.8	1.0	0.1	4.81	95.8
	Viktor H. [22]	Arria10	200	108.5K	1122	2195(M20k)	Float 32	27.5	202.0	0.9	0.09	–	–
	Deniel P. [23]	XCKU040	143	139.9K	839	384(36 kb)	Fixed 16	30.9	180.0	1.5	0.15	3.87	46.51
	Ours	Stratix10	150	67.1K	422	1806(M20k)	Fixed 16	23.8	226.7	3.58	0.179	4.35	52.11
	^a CS-4PETs	Stratix10	150	125K	908	1508(M20k)	Fixed 16	12.5	432.4	3.17	0.158	7.02	61.59

^a Performance and resource consumption statistics for the three CNN networks using four 14 × 32 PETs.

that PE utilization remains high without significant degradation when handling different CNN models.

Furthermore, in the Intel Stratix 10 series development boards, each DSP can perform two 16-bit × 16-bit fixed-point multiplications in parallel. In contrast, on other Intel series and Xilinx FPGA development boards, a single DSP can only perform one 16-bit × 16-bit or 32-bit × 32-bit multiplication operation at a time. Additionally, in Xilinx FPGA boards, a 25-bit × 18-bit DSP can be decomposed into two 8-bit × 8-bit multipliers. Therefore, compared to 16-bit accelerators, 8-bit accelerators achieve twice the overall DSP efficiency. Thus, the working frequency is partially determined by the FPGA device utilization. Therefore, the normalized DSP efficiency for different designs is computed as follows:

$$\text{Normalized DSP efficiency} = \frac{GOPS/DSP}{\alpha \times (Freq \div 100 \text{ MHz})} \quad (5)$$

For 16-bit or 18-bit accelerators, $\alpha = 1$ and for 8-bit accelerators or 16-bit accelerators on Stratix 10 series development boards, $\alpha = 2$. When comparing our design with other solutions based on AlexNet, although our design utilizes the maximum number of DSPs, it achieves the highest DSP utilization rate. Moreover, we have realized an energy efficiency ratio of 65.23 GOPS/W while operating at a relatively low power consumption of 7.15 W. Our VGG16 accelerator achieves a DSP efficiency of 3.97 and a normalized DSP efficiency of 0.198. This is slightly lower than the result in [18], as the platform used in that design has only 1518 DSPs, while the number of multipliers exceeds 1518, with the remaining multipliers implemented by logical elements. Therefore, the normalized DSP efficiency reported in that design is not effectively valid. In VGG16, the scheme proposed by Huang M et al. achieved an energy efficiency of 185.4 GOPS/W. However, this result

was obtained under the Fixed 8 data type. In the comparison with other designs on YOLOv3-Tiny, although our design utilizes a relatively large amount of storage resources, it achieves relatively high throughput and energy efficiency ratio with the smallest number of DSPs. Compared to other CNN accelerators listed in Table 1, our design offers a normalized DSP efficiency that is 1.06× to 2.22× higher.

4. Conclusion

In this paper, we propose a CNN accelerator based on CS dataflow. To mitigate the bandwidth overhead caused by repeated data reads, we design a CS dataflow scheme that excels in ifmap multiplexing and generalizability. For the CS dataflow, we propose an MRSA architecture optimized for pipeline performance through precise scheduling of the PEs, achieving cyclic splicing and latency overlapping. Experimental results on the forward inference networks of AlexNet, VGG16, and YOLOv3-Tiny demonstrates that the average utilization of the PEs in the convolutional layers can reach 86.5%, 98.8%, and 94.9%.

CRedit authorship contribution statement

Xiaojun Zhang: Conceptualization, Methodology. **Xin Song:** Software, Validation, Writing – original draft. **Shuguang Liu:** Methodology, Resources. **Yue Liu:** Writing – review & editing. **Jian Gao:** Supervision, Conceptualization, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by Natural Science Foundation of Shandong Province for Innovation and Development Joint Funds under Grant ZR2022LZH001.

Data availability

No data was used for the research described in the article.

References

- [1] Y.-H. Chen, J. Emer, V. Sze, Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks, in: ACM/IEEE 43rd Annual International Symposium on Computer Architecture, ISCA, IEEE, Seoul, South Korea, 2016, pp. 367–379.
- [2] Y.-H. Chen, T. Krishna, J.S. Emer, V. Sze, Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks, *IEEE J. Solid-State Circuits* 52 (1) (2017) 127–138.
- [3] M. Yin, X. Cui, F. Wei, H. Liu, Y. Jiang, X. Cui, A reconfigurable FPGA-based spiking neural network accelerator, *Microelectron. J.* 152 (2024) 106377.
- [4] H. Yu, D. Zhang, Y. Yang, S. Chen, Z. Li, Design of lightweight on-chip one-dimensional convolutional neural network accelerators for edge-end chips, *Microelectron. J.* 157 (2025) 106570.
- [5] Z. Du, R. Fasthuber, T. Chen, P. Jenne, L. Li, T. Luo, X. Feng, Y. Chen, O. Temam, ShiDianNao: Shifting vision processing closer to the sensor, in: Proceedings of the 42nd Annual International Symposium on Computer Architecture, ACM, Portland Oregon, 2015, pp. 92–104.
- [6] N.P. Jouppi, et al., In-datacenter performance analysis of a tensor processing unit, in: Proceedings of the 44th Annual International Symposium on Computer Architecture, ACM, Toronto ON Canada, 2017, pp. 1–12.
- [7] S. Ryu, Y. Oh, J.-J. Kim, Mobileware: A high-performance mobilenet accelerator with channel stationary dataflow, in: 2021 IEEE/ACM International Conference on Computer Aided Design, ICCAD, IEEE, 2021, pp. 1–9.
- [8] Y. Zhao, X. Chen, R. Xu, J. Wei, J. Lu, B. Yuan, A research and design of lightweight convolutional neural networks accelerator based on systolic array structure, in: 6th International Conference on Robotics and Artificial Intelligence, ACM, Singapore Singapore, 2020, pp. 241–247.
- [9] B. Huang, Y. Huan, H. Chu, J. Xu, L. Liu, L. Zheng, Z. Zou, IECA: An in-execution configuration CNN accelerator with 30.55 GOPS/mm² area efficiency, *IEEE Trans. Circuits Syst. I. Regul. Pap.* 68 (11) (2021) 4672–4685.
- [10] S.H. Chua, T.H. Teo, M.A. Tiruye, I.-C. Wey, Systolic array based convolutional neural network inference on FPGA, in: 2022 IEEE 15th International Symposium on Embedded Multicore/Many-Core Systems-on-Chip (MCSoc), IEEE, Penang, Malaysia, 2022, pp. 128–133.
- [11] Y. Parmar, K. Sridharan, A resource-efficient multiplierless systolic array architecture for convolutions in deep networks, *IEEE Trans. Circuits Syst. II: Express Briefs* 67 (2) (2020) 370–374.
- [12] M.P. Véstias, R.P. Duarte, J.T. De Sousa, H.C. Neto, A fast and scalable architecture to run convolutional neural networks in low density FPGAs, *Microprocess. Microsyst.* 77 (2020) 103136.
- [13] H. Li, L. Gong, C. Wang, X. Zhou, A flexible dataflow CNN accelerator on FPGA, in: IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW), IEEE, Bangalore, India, 2023, pp. 302–304.
- [14] T. Adiono, A. Putra, N. Sutisna, I. Syafalni, R. Mulyawan, Low latency YOLOv3-tiny accelerator for low-cost FPGA using general matrix multiplication principle, *IEEE Access* 9 (2021) 141890–141913.
- [15] Y. Liu, Y. Ma, B. Zhang, L. Liu, J. Wang, S. Tang, Improving the computational efficiency and flexibility of FPGA-based CNN accelerator through loop optimization, *Microelectron. J.* 147 (2024) 106197.
- [16] J. Shan, M.T. Lazarescu, J. Cortadella, L. Lavagno, M.R. Casu, CNN-on-AWS: Efficient allocation of multikernel applications on multi-FPGA platforms, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 40 (2) (2021) 301–314.
- [17] J. Li, K.-F. Un, W.-H. Yu, P.-I. Mak, R.P. Martins, An FPGA-based energy-efficient reconfigurable convolutional neural network accelerator for object recognition applications, *IEEE Trans. Circuits Syst. II: Express Briefs* 68 (9) (2021) 3143–3147.
- [18] Y. Zhang, H. Wang, Z. Pan, An efficient CNN accelerator for pattern-compressed sparse neural networks on FPGA, *Neurocomputing* 611 (2025) 128700, <http://dx.doi.org/10.1016/j.neucom.2024.128700>.
- [19] Y. Ma, Y. Cao, S. Vrudhula, J.-s. Seo, Optimizing the convolution operation to accelerate deep neural networks on FPGA, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 26 (7) (2018) 1354–1367.
- [20] M. Huang, Y. Liu, C. Man, K. Li, Q. Cheng, W. Mao, H. Yu, A high performance multi-bit-width booth vector systolic accelerator for NAS optimized deep learning neural networks, *IEEE Trans. Circuits Syst. I. Regul. Pap.* 69 (9) (2022) 3619–3631.
- [21] A. Ahmad, M.A. Pasha, G.J. Raza, Accelerating tiny YOLOv3 using FPGA-based hardware/software co-design, in: IEEE International Symposium on Circuits and Systems, ISCAS, IEEE, Seville, Spain, 2020, pp. 1–5.
- [22] V. Herrmann, J. Knapheide, F. Steinert, B. Stabernack, A YOLO V3-tiny FPGA architecture using a reconfigurable hardware accelerator for real-time region of interest detection, in: 25th Euromicro Conference on Digital System Design, DSD, IEEE, Maspalomas, Spain, 2022, pp. 84–92.
- [23] D. Pestana, P.R. Miranda, J.D. Lopes, R.P. Duarte, M.P. Vestias, H.C. Neto, J.T. De Sousa, A full featured configurable accelerator for object detection with YOLO, *IEEE Access* 9 (2021) 75864–75877.
- [24] S. Lee, D. Kim, D. Nguyen, J. Lee, Double MAC on a DSP: Boosting the performance of convolutional neural networks on FPGAs, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 38 (5) (2019) 888–897.